

Laboratorio 2 – Flota IoT Central y generación de datos

Python en máquina virtual (Azure) + ESP32 Arduino en Wokwi

Escenario aplicado: UNAB-Ambiental

José Alejandro Téllez Prada – U00060479

Docente: Javier Pinzón

Bucaramanga, Santander

Septiembre de 2026

1. Objetivos

- Reutilizar la plantilla `consola-unab-ambiental` del Laboratorio 1 para levantar una flota de 8 dispositivos.
- Generar telemetría real desde un nodo Python corriendo en una máquina virtual de Azure.
- Generar telemetría real desde un ESP32 simulado (Wokwi) con firmware Arduino.
- Configurar una regla de alerta por umbral y comparar los tres orígenes de datos: simulador nativo, Python-VM y Wokwi.

2. Despliegue en la nube: máquina virtual

Para el nodo Python se desplegó una máquina virtual en Azure (`vmjose1`), evaluando distintas regiones disponibles bajo la suscripción de estudiante. Se eligió la región **Chile Central** por ser la más apta en latencia y disponibilidad de cupo para el tamaño de VM requerido. El tamaño seleccionado fue **Standard B2ats v2** (2 vCPUs, 1 GiB de RAM, arquitectura ARM64), una opción económica (aprox. \$9 USD/mes) que permite dejar la VM encendida de forma continua sin agotar el crédito de estudiante. El sistema operativo es Ubuntu 24.04 LTS.

3. Inventario de la flota (8 dispositivos)

Cantidad	Tipo	Generación de datos
6	Simulados IoT Central	Simulador nativo de la aplicación
1	Python en VM (<code>23z8rpgm6s4</code>)	Script <code>python-vm-01.py</code> en la VM de Azure
1	ESP32 Arduino (<code>esp32-wokwi-01</code>)	Sensores virtuales en Wokwi for VS Code

4. Nodo Python en la VM

El script `python-vm-01.py` corre dentro de la VM usando `azure-iot-device` sobre `asyncio`: se aprovisiona vía DPS, se conecta a IoT Central y publica *Temperature*, *Humidity* e *Illuminance* cada

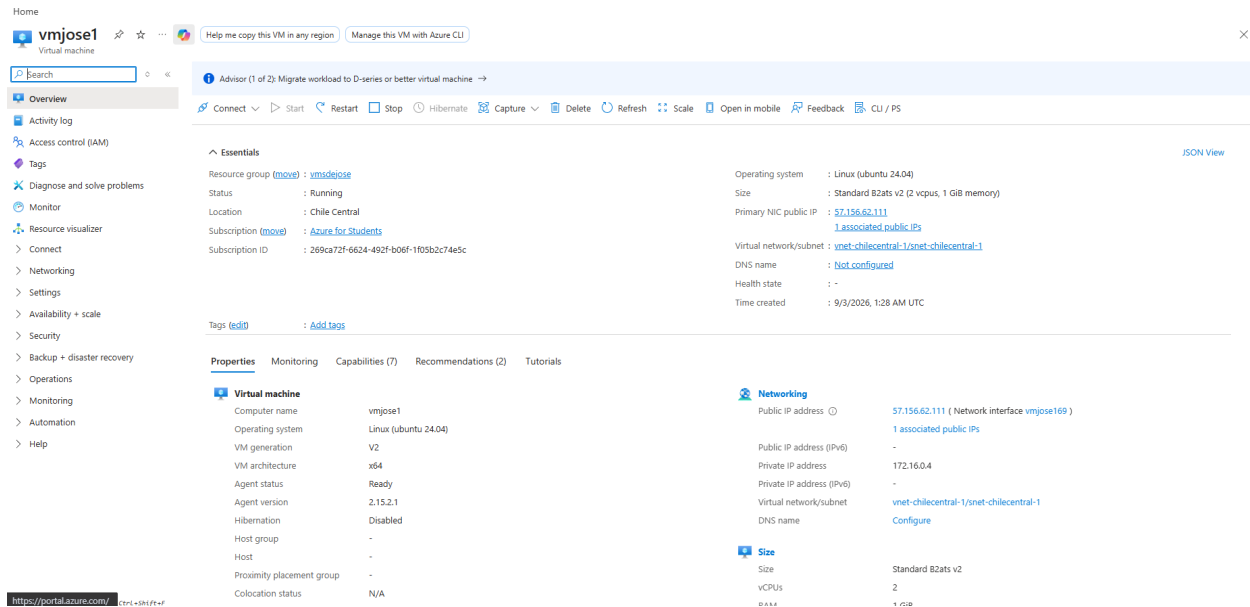


Figura 1: Máquina virtual `vmjose1` en la región Chile Central, tamaño Standard B2ats v2.

10 segundos, generadas por una caminata aleatoria acotada a los rangos operativos del DTDL (16–32 °C, 35–70 %RH, 100–800 lux). Además atiende la property writable `Set_temp_hvac` y los comandos `Encender_hvac` y `force_reading`.

Sobre ese mismo script se construyó un panel web (Flask, hilo independiente del loop de `asyncio`) que funciona como "panel del profesor": muestra el semáforo de estado, las 3 variables en vivo, un historial de las últimas 40 lecturas, y permite desde el navegador cambiar el setpoint del HVAC, forzar una lectura, o forzar una temperatura alta para disparar la regla de alerta por correo.

El panel queda disponible en vivo (mientras la VM esté encendida) en:

<http://57.156.62.111:5000>

Nota de seguridad: este panel expone acciones de control (cambiar setpoint, forzar alertas) sin autenticación. El puerto se dejó abierto de forma intencional en el grupo de seguridad de red (NSG) para que el docente pueda revisarlo directamente por IP pública durante la evaluación; una vez calificado el laboratorio, se recomienda cerrarlo o añadir autenticación básica, ya que cualquiera con la URL podría alterar el estado del dispositivo o disparar notificaciones de correo.

5. ESP32 en Wokwi

El firmware del ESP32 (basado en el ejemplo oficial *Azure SDK for C* adaptado al DTDL propio) lee un sensor DHT22 (Temperature, Humidity) y un potenciómetro mapeado a Iluminance, y publica las 3 variables cada 10 segundos usando DPS + MQTT sobre TLS.

Inicialmente se intentó trabajar en `wokwi.com`, pero la plataforma presentó saturación ("server busy") de forma reiterada, por lo que se migró a la extensión **Wokwi for VS Code**, que corre la simulación de forma local e incluye un *IoT Gateway* privado con el que el ESP32 simulado sí logra conectarse a Azure real. Esto requirió instalar un toolchain de compilación (`arduino-cli` con el core `esp32:esp32`), ya que la extensión solo ejecuta un firmware ya compilado.

```

C:\Users\jose>ssh jtellez312@57.156.62.111
The authenticity of host '57.156.62.111 (57.156.62.111)' can't be established.
ED25519 key fingerprint is SHA256:ixXC2AF9VvvcJ+0JNmM1l1P2NaoGL8sIwDqSum7Pm4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Host key verification failed.

C:\Users\jose>ssh jtellez312@57.156.62.111
The authenticity of host '57.156.62.111 (57.156.62.111)' can't be established.
ED25519 key fingerprint is SHA256:ixXC2AF9VvvcJ+0JNmM1l1P2NaoGL8sIwDqSum7Pm4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '57.156.62.111' (ED25519) to the list of known hosts.
jtellez312@57.156.62.111's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1022-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Sep  3 01:34:23 UTC 2026

System load:  0.0               Processes:            116
Usage of /:   5.6% of 28.02GB   Users logged in:     0
Memory usage: 31%              IPv4 address for eth0: 172.16.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

jtellez312@vmjose1:~$ echo "hola"
hola
jtellez312@vmjose1:~$ |

```

Figura 2: Primera conexión SSH a la VM y verificación del acceso remoto.

Los comandos de la plantilla (*Encender_hvac*, *force_reading*) se adaptaron en el firmware para tener efecto real en la simulación: *Encender_hvac* alterna el LED rojo y reporta el estado del semáforo; *force_reading* envía una lectura inmediata fuera de la ventana de 10 segundos.

Nota: el dispositivo Wokwi solo aparece como *Connected* en IoT Central mientras la simulación está corriendo activamente; al cerrarla pasa a *Disconnected*, comportamiento esperado dado que no hay hardware físico sosteniendo la conexión.

6. Flota completa en IoT Central

La siguiente vista muestra, a modo representativo, uno de cada tipo de dispositivo sobre la misma plantilla consola-unab-ambiental: el nodo Python real (python-vm-01), un simulador nativo (Simulador-salon-1) y el ESP32 de Wokwi (ESP32 Wokwi).

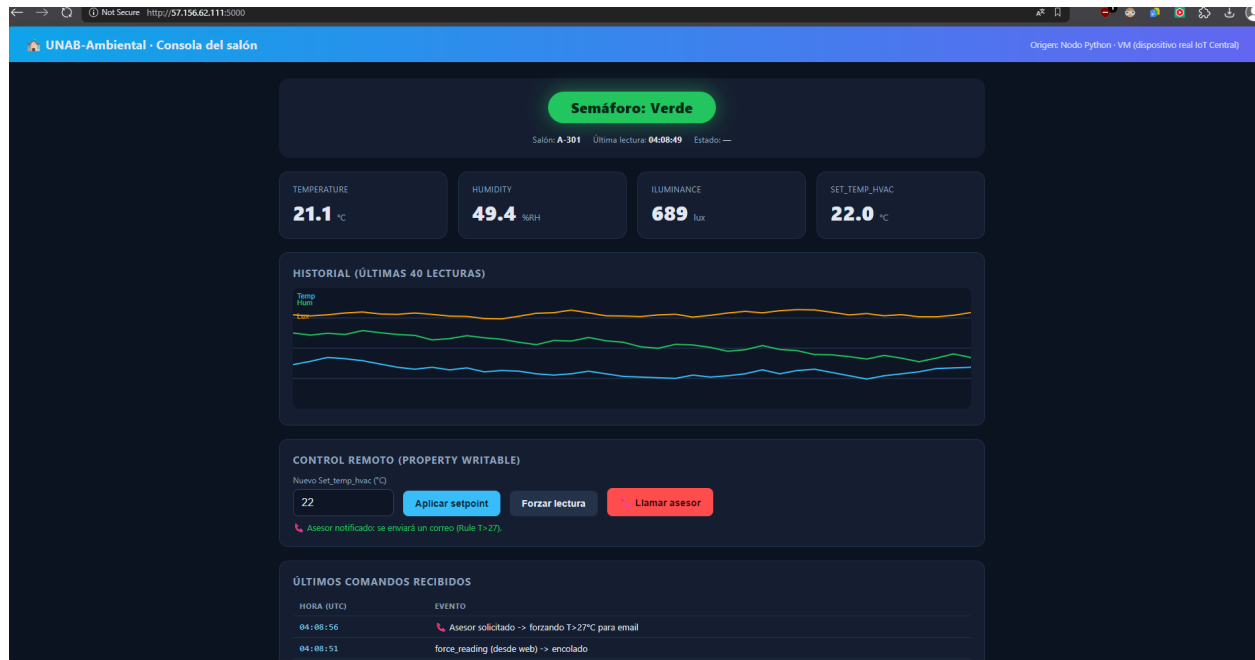


Figura 3: Panel web (Flask) del nodo Python: telemetría en vivo, semáforo, historial y control remoto.

7. Regla de alerta

Se configuró en Azure IoT Central la regla **Alerta Temperatura alta**: condición *Temperature* > 27 °C, acción de envío de correo a jtellez312@unab.edu.co con un mensaje describiendo la alerta. Esta regla puede dispararse manualmente desde el panel Flask con el botón "Llamar asesor", que fuerza una lectura por encima del umbral para efectos de prueba.

8. Repositorio

Todo el código (script Python, proyecto Wokwi, helpers de conexión a la VM y esta bitácora) está versionado en: https://github.com/joserraldo/Iot_learning

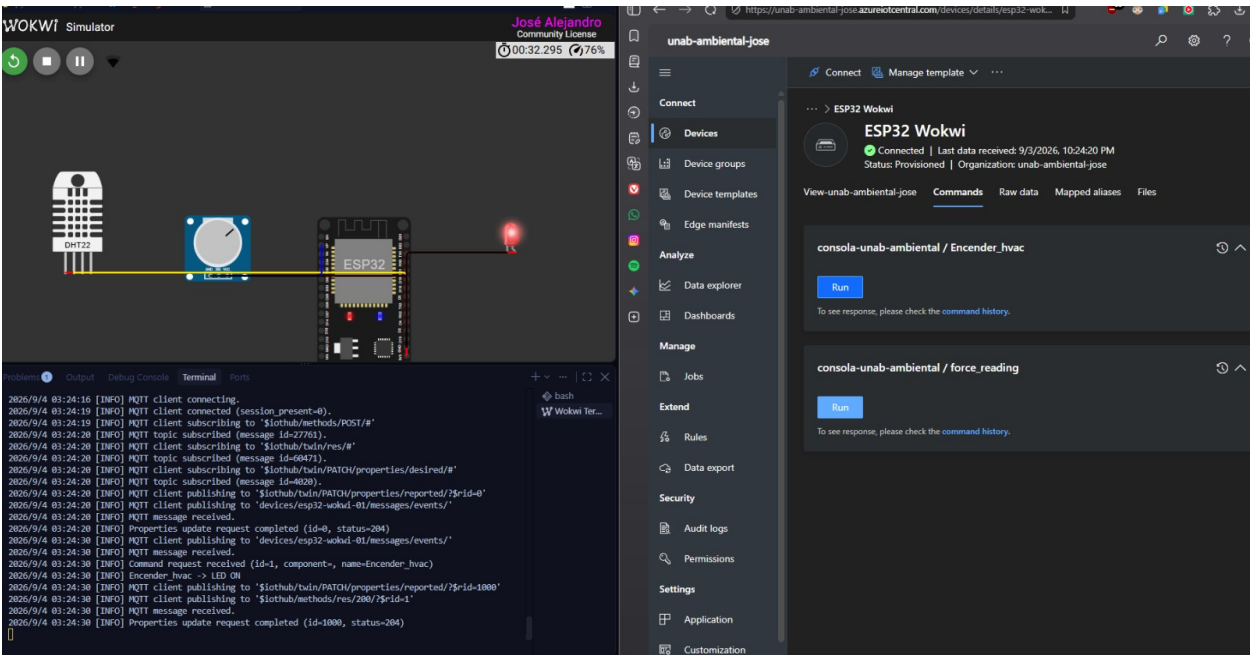


Figura 4: ESP32 simulado en Wokwi (izquierda) y panel de comandos en Azure IoT Central (derecha): *Encender_hvac* recibido, LED encendido y confirmación por MQTT.

Results (3 devices)				
Name	Device ID	Simulated	Device status	Organization
python-vm-01	23z8rpgm6s4	No	Provisioned	unab-ambiental-jose
Simulador-salon-1	29edu1h6730	Yes	Provisioned	unab-ambiental-jose
ESP32 Wokwi	esp32-wokwi-01	No	Provisioned	unab-ambiental-jose

Figura 5: Dispositivos de la flota sobre la plantilla consola-unab-ambiental: simulador, VM y Wokwi.

9. Tabla comparativa: simulador nativo vs Python-VM vs Wokwi

Origen	Cómo se genera el dato	Latencia	Facilidad de debug	Fallos de red
Simulador nativo	IoT Central genera datos sintéticos según el rango del DTDL, sin hardware.	Inmediata (dentro del portal)	Alta: logs integrados en el portal	Ninguno (interno a Azure)
Python-VM	Script en Ubuntu (VM), caminata aleatoria acotada a los rangos del DTDL, cada 10 s.	~1-2 s desde DPS hasta IoT Central	Media: requiere SSH + <code>run.log</code> con salida sin buffer	Ninguno en DPS/MQTT; puerto 5000 del dashboard bloqueado por firewall externo (mitigado con túnel SSH)
Wokwi (ESP32)	Firmware Arduino leyendo DHT22 + potenciómetro, cada 10 s vía Azure SDK for C.	Depende de la simulación local	Baja: debug por monitor serial a 115200 baudios	wokwi.com saturado ("server busy"); resuelto migrando a Wokwi for VS Code con gateway local

10. Lecciones aprendidas y limitaciones

- El motor de reglas de IoT Central evalúa umbrales de forma inmediata; la semaforización por ventana de tiempo (usada en el diseño teórico) tuvo que replicarse en la lógica del propio script Python, no en el portal.
- El panel local (Flask) es una vista de depuración, no reemplaza al gemelo digital real de Azure IoT Central; se debe evitar exponerlo sin autenticación si se abre el puerto públicamente.
- Wokwi en el navegador no es completamente confiable para trabajo sostenido (saturación del servicio); la alternativa local (Wokwi for VS Code + `arduino-cli`) fue más estable para lograr una conexión real a Azure.
- El uso de asistentes de IA en la terminal fue clave para resolver errores de compilación, configuración de pines en Wokwi, y automatizar la conexión remota a la VM (scripts de `paramiko` en `tools/`), acelerando la resolución de problemas de red y de firmware.